第四届开放原子大赛 · 2026 开源行业解决方案创新赛
赛道一：开源基础软件与解决方案

Riffnel_memguard

面向 AI 智能体的记忆错误调查与修正验证平台

Memory error investigation and correction verification for AI agents

文档类型	作品介绍文档（草稿 v0.1）
文档日期	2026 年 10 月 20 日
代码仓库	https://github.com/ChakesWu/Riffnel_memguard
开源协议	Apache-2.0
演示视频	【待补：视频链接】
团队与联系方式	【待补：团队名称、成员、邮箱】

本文所有性能数字均来自仓库中实际运行的基准与验证输出；示例数据全部为仿真，不对应任何真实机构、条款或个人；本项目不暗示任何机构的认可或背书。

目录

1 项目概述 3

2 背景与痛点 3

3 解决方案 5

4 技术架构 6

5 关键技术与实现要点 7

6 创新点 8

7 竞品分析 9

8 应用场景 10

9 验证与数据 11

10 开源与治理 12

11 商业模式与长期规划 13

12 团队与资源 13

13 风险与对策 14

附录 15

1 项目概述

一句话定位：Riffnel_memguard 不存储记忆。它接在现有智能体记忆库（Mem0 等，或自研存储）之上，在智能体答错时查出是哪条记忆导致、为什么错；生成可审批、可回滚的修正方案；再用可重现的测试验证「错误已消失、没有引入新错误」，并输出带校验链的证据报告。可以理解为记忆的「事故调查 + 修正验收」层。

要解决的问题	智能体的长期记忆会出错，并且错误跨会话持续影响回答：权威来源（制度、产品条款）更新后记忆仍是旧版本；删除请求只删了原始记录、摘要里仍有残留；记忆被写入恶意指令等。现有项目多解决「怎么存、怎么取、怎么防写入」，出事之后「如何定位责任记忆、受控修正、并证明修正有效」仍缺少完整流程。
我们做了什么	溯源链接（记忆 → 条款版本）、事故调查与六类错误归因（M1–M6）、非破坏性修正（预演、审批、应用、回滚）、验证（重放、回归、残留探测）、证据报告、记忆体检与来源更新后的重新验证；提供命令行、Web 界面与 Python 接口。
当前状态	v0.1 预览，已可运行：约 2500 行 Python 核心代码，93 项自动化测试，持续集成（Python 3.10、3.12、容器镜像）通过。适配器：JSON、SQLite、Mem0、Letta；另可导入 memwall 扫描结果。
验证情况	仿真基准 74 个用例；并在真实堆栈上验证（Mem0 + 本地向量库 + Pydantic AI 智能体 + DeepSeek 语言模型 + 真实中文嵌入模型）：条款过期场景在修复回归检查方式后 10 次全部通过（修复前为 6/10，过程见第 9 节），删除残留场景 5 次全部通过。样本小、数据为仿真，详见第 9 节及其限制。
一个真实发现	在 Mem0 开源版 2.2.1 上实测：用它自带的 `delete` 删除一条记忆后，被删文字的完整内容仍保留在其历史库（history.db）中。本项目把这类残留纳入探测并清除。
开源与合规	Apache-2.0；包含贡献指南、行为准则、治理、安全政策、路线图、更新日志与第三方许可证清单；默认离线运行、无遥测。
与同类项目的关系	溯源、审计、来源漂移检测、回滚、残留检查，现有开源项目各自已有一部分（第 7 节逐项核实）。本项目的差异是把它们组合为面向事故的「调查、修正、验证」流程，并以带有效期的条款版本为基准；这是组合创新，不是单项技术的首创。

2 背景与痛点

2.1 智能体记忆成为新的故障来源

带长期记忆的智能体（客服、助手、业务流程智能体）把对话中的事实、摘要、偏好写入记忆库，之后跨会话取用。记忆一旦出错，错误会反复出现，且不会像一次提示词错误那样「重启即消失」。常见的错误来源有六类：

- 权威来源更新后，记忆里仍是旧版本（过期）；
- 同一事项存在互相矛盾的多条记忆（冲突）；
- 属于 A 用户、A 产品、A 地区的记忆被用在 B 身上（串用）；
- 没有来源依据的内容被写进记忆，例如模型自行概括或幻觉（无来源写入）；

- 用户要求删除的个人信息，原始记录删了，摘要、向量或历史库里仍有残留（应删未删）；
- 被注入的恶意内容写进记忆（投毒）。

2.2 两个典型事故（本项目的演示场景，数据为仿真）

场景	经过
过期（M1）	某银行信用卡「金卡免年费」条件由每年刷满 3 笔改为 6 笔（2026-07-01 生效）。智能体仍从旧记忆取值，对客户答「3 笔」。需要查出是哪条记忆、是否还有依赖它的摘要、改什么、改完如何证明。
应删未删（M5）	客户要求删除手机号。原始记录已删除，但一条回访摘要里仍写着号码，智能体被问到时照样说出。

2.3 一个真实框架里的残留

我们在 Mem0 开源版 2.2.1 上实测：`Memory.delete` 只删除活动记忆，被删文字的完整内容（新增、更新、删除三类记录）仍保留在它的 SQLite 历史库中。也就是说，即便「调用了删除接口并返回成功」，数据仍可能留在同一套框架的另一张表里。在 Letta 0.16.7 上我们看到另一种情形：删除单条归档记忆是硬删除（整库传储中找不到文本），但删除智能体并不会删除它的归档和归档记忆，文本仍留在数据库里。这类残留需要被主动探测，而不能默认删除成功。

2.4 行业与合规背景

- 个人信息删除。《中华人民共和国个人信息保护法》第四十七条规定了个人信息处理者应当主动删除个人信息的情形，以及个人请求删除的权利；同时规定了例外（如法定保存期限未届满、删除在技术上难以实现时，应停止存储和必要安全保护以外的处理）。本文据二手资料引用，未逐项核对正式法律文本，引用前应以全国人大或国家法律法规数据库的正式文本为准。来源见附录 B。
- 记忆与上下文投毒。OWASP 发布的「智能体应用十大风险（2026）」把「记忆与上下文投毒」列为 ASI06。本文据二手资料引用，未直接核对 OWASP 原文。

2.5 现有方案的缺口

现有开源项目已分别覆盖：记忆的存取与去重（如 Mem0）、写入前扫描与隔离（如 memwall）、溯源与审计日志（如 typed-recall、memory-blackbox）、来源漂移检测（如 agentmem、mneme）、删除证明与隔离探测（如 Ferryte，源码可见但非 OSI 开源）。缺口在于：出事之后，以权威来源的版本和有效期为基准，系统化地归因、受控修正、批量回归并出具可审计的证据，且能在离线、可复现、可开源自部署的条件下完成。

3　解决方案

3.1　核心思路：调查 → 修正 → 验证

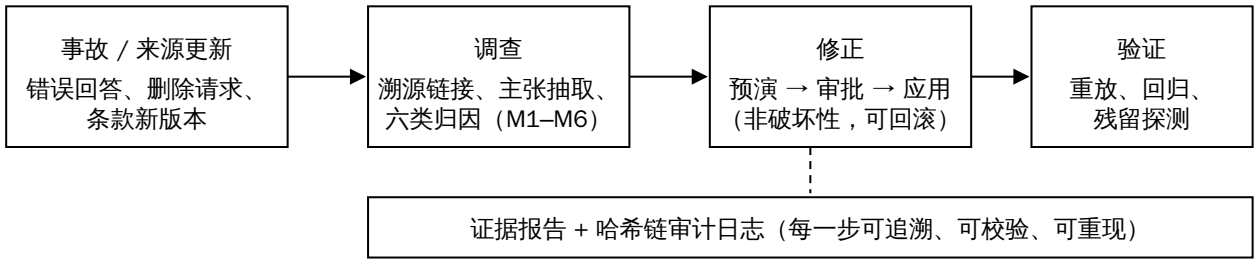


图 1　闭环流程

3.2　六类记忆错误与默认修正

代码	名称	判定信号	默认修正
M1	过期	记忆对应的条款版本在事故发生时已失效	停用旧记忆，按现行版本新增，清理派生摘要
M2	冲突	同一主题、同一作用域有多条有效记忆且取值不同，落败者不是被取代的旧版本	来源能裁定则停用落败者；裁定不了则隔离待人工复核
M3	串用	回答依据的记忆所属作用域与事故作用域不符	把记忆改回正确作用域
M4	无来源写入	记忆陈述规则类事实，取值却对应不到任何条款版本	隔离，交人工复核
M5	应删未删	已受理删除请求，但原始记录、摘要或历史库中仍能找到	删除并清理派生与历史残留，再探测
M6	投毒	含指令性内容（仅基础启发式），或由外部扫描工具标记	隔离

多类同时命中时，按「越具体、越可确定越优先」的顺序判定：M5 → M3 → M1 → M2 → M4 → M6。归因的前提是回答中的错误主张能与某条被取用的记忆对上；对不上时，系统给出「无法归因」，说明问题不在记忆层，应转向提示词或模型层处理。

3.3　设计原则

原则	含义
非侵入	通过适配器接入现有记忆库，只需在检索处记录取用了哪些记忆。
证据优先	每个结论都附记忆原文、条款原文、版本、时间与校验值。
非破坏性	默认「停用并新增」，保留历史；只有删除请求才物理删除，且审计日志只记录对象 ID 与哈希，不记录被删内容。

人在回路	修正方案先预演再审批，审批人写入审计链；未审批不能应用。
离线、确定性	默认不联网、无遥测；规则加字符 n-gram 检索；语言模型与真实嵌入器是可选项。
可重现	修正前后快照校验值、参数、生效日期写入报告。

4 技术架构

4.1 分层结构

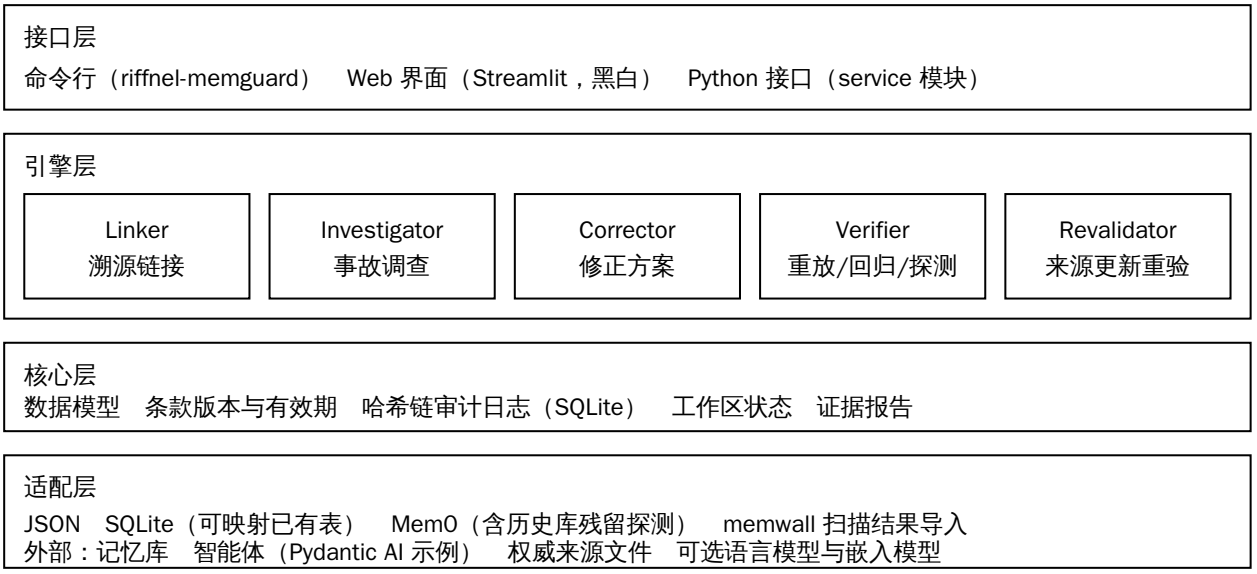


图 2 分层架构

4.2 模块与数据

模块	职责
溯源链接（Linker）	把记忆连到它依赖的条款版本：显式标注（置信度 1.0）、主题键比对（0.9）、语义相似度（取得分）。取值对不上任何版本的记忆列为 M4 候选。并提供影响分析：条款版本 → 记忆 → 派生记忆 → 测试题。
事故调查（Investigator）	从回答中抽取主张（数字加单位，支持阿拉伯数字与中文数字），按事故发生时间找出当时有效的条款版本，在被取用的记忆里找出取值相同者，按判定顺序归类并附证据。
修正（Corrector）	按类别生成动作（停用、新增、改作用域、删除、清理派生、隔离），支持预演、审批、应用、回滚；每步写入审计链。
验证（Verifier）	重放原事故；沿溯源图选出相关测试题批量回归，并检查「修正前通过的题仍通过」；删除请求做残留探测。
残留探测	精确（字符串）、变形（全角、空格、连字符、中文数字、邮箱拼写混淆）、行为（让智能体回答探测问题）三层为硬性关卡；语义层只提示「待人工复核」。
审计日志	只追加；每条事件带前一条的 SHA-256 哈希，篡改可被检出；只记录对象 ID 与哈希，不记录被删内容。

适配器	统一接口：列出、读取、检索、新增、停用、隔离、恢复、删除、改作用域、快照；可选 `traces()` 暴露存储在活动记忆之外的残留（如 Mem0 历史行）。

4.3 技术栈

Python 3.10 及以上；pydantic（数据模型）、Typer 与 Rich（命令行）、scikit-learn（字符 n-gram TF-IDF 检索）、SQLite（审计日志与参考存储）、Streamlit（界面，可选）、pytest（测试）。可选组件：Mem0（Apache-2.0）、Pydantic AI（MIT）、fastembed（Apache-2.0）及 BAAI/bge-small-zh-v1.5 嵌入模型（Hugging Face 模型页标注 MIT）。第三方依赖的许可证已核实至直接依赖，传递依赖尚未逐项核实（见仓库 THIRD_PARTY_LICENSES.md）。

4.4 部署方式

三种方式均无需联网、无需密钥即可运行演示：（1）`pip install -e ".[dev]"` 后使用命令行；（2）`docker compose up --build` 启动 Web 界面（持续集成中已构建镜像并在容器内运行两个演示）；（3）作为 Python 库调用 service 模块。接入真实模型与嵌入器通过环境变量配置，不改代码。

5 关键技术与实现要点

5.1 以条款版本和有效期为基准的归因

条款（如「金卡免年费条件」）带有多个版本，每个版本有取值与有效期区间。事故调查按事故发生的时间而不是「今天」判断当时的有效值，因此同一条旧记忆在 2026-05 的事故中是正确的，在 2026-08 的事故中是过期的（基准集中的「历史有效」用例专门检验这一点）。来源更新后，无需等事故，系统沿溯源链接找出仍依赖旧版本的有效记忆并生成修正方案。

5.2 非破坏性修正与审批

过期记忆被「停用」而不是删除，并新增现行版本、保留取代关系；派生摘要一并清理。方案先以预演（diff）展示将发生的变更；方案状态被强制为 草稿 → 已审批 → 已应用（或应用中断）→ 已回滚，只有审批后才能应用，已应用的方案不能再审批或重复应用。审批人名单（`RIFFNEL_APPROVERS`）与「起草人不能审批自己的方案」（`RIFFNEL_FOUR_EYES`）可配置，但这是本地信任而不是身份认证。应用按动作逐个写日志：中途失败时方案停在 `apply_failed`，再次应用从断点继续（动作均幂等），回滚只撤销已完成的部分；这不是事务，各记忆库不提供跨多条记忆的事务。停用、隔离、新增、改作用域均可回滚，删除不可回滚且会明确提示。已有一条符合来源的记忆时不重复新增。

5.3 删除验证与历史库残留

删除请求的验证不信任「删除接口返回成功」。探测同时检查活动记忆、派生摘要、存储自带的历史行（trace），并让智能体回答探测问题以检查行为层。验证中另用不依赖本项目探测器的方式直接扫描数据目录原始字节，作为独立旁证。删除请求的目标值只暂存在权限 0600 的文件中，应用删除方案时清除工作区状态里所有含目标值的文本并删除含目标值的报告，验证通过后丢弃目标值；这一点是作者对

照自己的说法检查后补上的（此前工作区状态会保留事故记录里的号码）。语义层在基准中误报较多（见第 9 节），因此只作人工复核提示。

5.4 与真实系统接入

- Mem0 适配器：读写 Mem0 记忆，修正状态存放在其元数据中；Mem0 默认向外发送匿名遥测，本项目在自带的构造函数中强制关闭。
- Letta 适配器：把 Letta 智能体的归档记忆当作记忆库。Letta 的归档记忆不能修改，所以字段存放在标签中，停用或改作用域通过「先写新条目、再删旧条目」完成，逻辑 id 不变；列出使用游标分页。在本机 Letta 0.16.7 上两个演示场景各跑通一次，6 项集成测试通过，并用 pg_dump 独立检查数据库。测试环境不是受支持的部署方式（需还原发布当日依赖、PostgreSQL，且需手动建表），持续集成中没有 Letta 服务端，相关测试被跳过。
- SQLite 适配器：可映射已有表的列名，校验表名防注入；没有状态列时无法停用，会明确报错。
- 企业客服智能体示例：基于 Pydantic AI，固定策略（必须先调用检索工具、只依据返回内容回答、没有结果就拒答），记录每次检索，并生成事故。模型由环境变量选择，离线时使用确定性替身（不是语言模型）。
- 任意智能体接入（`--agent-cmd`）：外部智能体只需从标准输入读一行 JSON、向标准输出写一行 JSON（回答与取用的记忆 id），即可被调查、重放与回归，与语言无关；已用 Node.js 写的智能体完整跑通。命令不经过 shell 执行，但以本机权限运行，属受信任配置。Web 界面也可使用，但默认关闭（网页执行命令等于开放远程执行），需在本机可信环境显式启用。
- memwall 导入：把 memwall 扫描报告导入为 M6 发现，并导出记忆供其扫描。导入器按 memwall 0.6.1 的实际输出编写。

6 创新点

先说明边界：本项目的各个组成部分，现有开源项目大多已有某种形式的实现。我们声称的创新是组合与流程，而不是任何单项技术的首创。

序	创新点	内容	与现有项目的关系
1	条款版本与有效期作为基准	按事故发生时间判定当时有效的条款版本与作用域，据此归因；来源更新触发影响分析与重新验证。	agentmem、mneme、MemGuard（ac12644）有来源变化检测，但页面未写带有效期的版本判定。
2	面向事故的完整流程	六类根因分类；带审批和回滚的非破坏性修正；沿溯源图批量回归，并检查「修正前通过的题仍通过」。	memory-blackbox 已有追溯、回滚与重跑；Ferryte 有 `why --replay`。审批流程与批量回归是本项目的组合点。

3	面向历史库残留的删除验证	探测范围含派生摘要与存储自带历史行；行为层让智能体回答探测问题；独立字节扫描作旁证；语义层降为提示。	memwall、inspeximus、Ferryte 都有残留或删除检查。差异在于覆盖历史行与行为层的组合。
4	真实堆栈上的可复现验证	离线、确定性的默认模式；同一套流程在 Mem0、本地向量库、真实语言模型与真实嵌入模型上跑通，并记录失败与修复过程。	——

在真实模型验证中，我们发现并修复了仿真无法暴露的缺陷：回归检查用死板的子字串比对，而真实模型会把「6笔」写成「6 笔」或「**6 笔**」，导致第一批 10 次运行中回归仅通过 6 次；修复比对方式后重新测量为 10 次全部通过（第 9 节）。

7 竞品分析

核实日期 2026-10-20，方法是访问各项目页面，只记录页面写明的内容；页面未写的记为「页面未提及」，不代表该项目没有此能力。各项目链接与完整记录见仓库 docs/COMPETITORS.md。

项目	许可证	页面所述能力	页面未提及 / 备注
Mem0	Apache-2.0	记忆层；开源版提供读取、更新、删除、历史等接口；新算法为只新增的抽取	冲突处理与来源链接未见说明。本项目在其 2.2.1 版上实测历史库残留
Letta	Apache-2.0	有状态智能体平台，记忆持久化；归档记忆可新增、列出、搜索、删除（不可修改）	README 未写记忆调试功能。本项目已实现归档记忆适配器并在本机 0.16.7 上测试；观察到删除智能体不删除其归档
memwall	MIT	扫描记忆（规则 MW001–MW016）、MCP 写入网关与隔离、追加式审计、按字面标识符的残留检查	自述无法证明改写或语义层面的删除；未见修正后重放与回归。本项目与其互补并可导入其结果
typed-recall	Apache-2.0	类型化边图谱、`trace(answer)` 溯源、`forget(node_id)` 精准遗忘、追加式日志	遗忘是否真的清除的验证方法页面未提及
Aegis Memory	Apache-2.0	不可变事件日志、旧记忆弃用但可查、HMAC 签名的来源记录	删除与擦除页面未提及
memory-blackbox	Apache-2.0	追加式账本、根因追溯、回滚（只追加不删除）、回滚后重跑有害动作	自述早期开发，接口未稳定
MemGuard（ac12644）	Apache-2.0	按来源 URL 重新抓取比对、信任分数、低分自动隔离、审计链	同名项目，因此本项目改名为 Riffnel_memguard；页面显示 3 stars

Ferryte	BSL 1.1（源码可见，非 OSI 开源）	自动挂接 Mem0、Chroma、Qdrant、pgvector；`why --replay`；隔离探测；带 Ed25519 签名的删除证明；血缘图；GitHub Action	自述 Pre-launch；未见修正方案审批与回滚、条款有效期版本、批量回归（页面未提及）。它在接入真实系统和删除证明上比本项目更完整

结论：最接近的是 Ferryte 与 memory-blackbox。本项目的定位不是取代它们，而是在「条款版本为基准、带审批与回滚的修正、批量回归、离线确定性、Apache-2.0 开源」这一组合上提供另一种选择；在接入真实系统的广度上，本项目目前明显落后于 Ferryte。

8 应用场景

8.1 场景一：产品条款更新后的智能体记忆过期（仿真）

1. 事故：智能体回答「金卡免年费需要每年刷满 3 笔」，而现行条款（2026-07-01 起）是 6 笔；
2. 调查：定位责任记忆（M1），同时发现一条依赖它的旧摘要，置信度来自溯源链接；
3. 修正：预演展示「停用两条旧记忆、新增一条现行版本」；经审批后应用；
4. 验证：重放原问题得到 6 笔；批量回归其他题没有退化；审计链完整；
5. 输出：含记忆原文、条款原文、版本、修正前后快照校验值的证据报告。

8.2 场景二：删除请求后的残留（仿真，真实记忆库）

1. 用 Mem0 自带的删除接口删除原始手机号记录；
2. 探测：精确层在历史行中找到号码，变形层在「138 0000 0000」的摘要中找到，行为层发现智能体被问到时说出了号码；
3. 修正：删除摘要，并清除历史行后压缩数据库文件；
4. 验证：探测通过，智能体不再说出；另用独立的原始字节扫描确认数据目录中不再含目标。

8.3 可延伸的使用方式

方式	说明
上线前验收	在持续集成中运行体检与回归，作为智能体记忆变更的放行依据。
来源更新后巡检	制度或条款发布新版本后运行体检，找出仍依赖旧版本的记忆。
个人信息删除核验	对删除请求做多层探测并保留带校验的证据，不记录被删内容。
事故复盘	从一次错误回答出发，得到责任记忆、证据与可审批的修正方案。

以上均为使用方式的设想；本项目尚未在任何真实企业系统中部署，也没有真实客户。

9 验证与数据

读数前提：基准用例由项目维护者自行设计，事故回答由有问题的记忆脚本化生成；真实模型验证样本很小。这些数字说明流程在这些情形下能走通，不代表真实环境的准确率。所有数字由仓库中的命令与记录文件产生。

9.1 仿真基准（74 个用例，命令：riffnel-memguard benchmark）

指标	结果
归因：责任记忆被找到	40/40
分类：类别完全正确	40/40
误报：无错用例（含无法归因）未产生发现	10/10
溯源：过期记忆被链接到正确的旧版本	23/23
修正后重放通过	40/40
回归无退化	40/40
删除探测：有残留的常规用例被硬性关卡探测到、准确定位、修正后清空	各 10/10
删除探测：无残留用例被硬性关卡误判（含相差一位的近似号码）	12/12 无误报
困难用例（中文数字、邮箱拼写混淆）被探测并定位	2/2
语义层（仅提示）命中有残留用例 / 误报无残留用例	3/12 / 6/12

事故类的 100% 是在设计好的用例上取得的，不应被解读为准确率。语义层的表现（对无残留用例提示 6/12，对有残留用例仅提示 3/12）是我们把它降为「待人工复核」提示的依据；用例重新生成前同一份探测代码曾得到不同的误报数，说明该层对数据敏感。

9.2 真实堆栈验证（2026-10-20）

记忆库为 Mem0 2.2.1 加本地 Qdrant；智能体为 Pydantic AI；语言模型为 DeepSeek deepseek-chat；嵌入模型为 BAAI/bge-small-zh-v1.5（经 Mem0 的 fastembed 嵌入器）。每次运行使用全新的记忆库目录。

批次与场景	结果
条款过期，10 次，离线哈希嵌入，修复前	事故重现、归因、重放均为 10/10；回归 6/10（4 次失败均为检查方式缺陷）
条款过期，10 次，离线哈希嵌入，修复后	事故重现、归因、重放、回归均为 10/10
删除残留，5 次，离线哈希嵌入	修正前探测发现残留 5/5（其中真实模型说出手机号 5/5）；修正后探测通过、模型不再说出、原始字节不含目标均为 5/5

条款过期，10 次，真实嵌入模型	事故重现、归因、重放、回归均为 10/10
删除残留，5 次，真实嵌入模型	同上，均为 5/5

观察：真实模型会自行改写检索查询，使取回的三条记忆出现不同排序（10 次运行中出现 3 种），归因与修正在各种排序下都成立。修复回归比对是在看到第一批失败之后做的，第二批是修复后的重新测量，不是对同一批结果的重新判分。完整记录见仓库 benchmark/REAL_MODEL_RUNS.md。

9.3 已知限制（如实）

- 所有数据为仿真示例，没有接入任何真实企业系统；
- 审批人是调用者输入的名字（本地信任，名单与四眼规则只是约束），不是身份认证；审计 HMAC 是对称的，持有密钥者可伪造；应用方案不是事务；文件锁未在 Windows 上测试，不适合网络文件系统；目标值在验证通过前以明文暂存于 0600 文件中；
- 真实模型验证只覆盖 M1 与 M5，只用了一个模型和一个嵌入模型，每个场景 5 到 10 次，模型输出有随机性；
- M2、M3、M4、M6 只在仿真基准中验证；M6 仅为基础启发式；
- 主张抽取目前只认「关键词 + 数字 + 单位」（支持中文数字到 999），不处理更复杂的表述；
- 溯源链接的语义相似度阈值（0.2）是未经验证的起始值；
- Mem0 的接口没有偏移或游标：适配器逐步放大读取上限直到取完，到上限会报错而不是静默截断；单条查找在本地嵌入式向量库中仍是线性扫描（开发机实测 1500 条约 0.26 秒，非基准）；只在 mem0ai 2.2.x 上测试；
- 原始字节扫描只覆盖本次工作目录，SQLite 空闲页是否残留取决于具体构建，不构成通用保证；
- Letta 适配器只在本机 Letta 0.16.7 + letta-client 1.10.x 上测试：测试服务端需还原发布当日依赖、PostgreSQL，并手动建表，不是受支持的部署方式；每个场景只跑通一次演示，未做真实模型与真实嵌入器的多次验证；只覆盖归档记忆；持续集成无 Letta 服务端，相关测试被跳过；
- Docker 映像由 GitHub 持续集成构建并运行演示，作者本地环境没有 Docker 服务。

10 开源与治理

项目	现状
开源协议	Apache-2.0（含专利授权）。
仓库文件	README、LICENSE、CONTRIBUTING、CODE_OF_CONDUCT、GOVERNANCE、SECURITY、ROADMAP、CHANGELOG、THIRD_PARTY_LICENSES。
贡献流程	Issue → 讨论 → Pull Request → 审查 → 合并；新增依赖须说明许可证；示例数据必须标明「仿真」。
响应承诺	Issue 与 Pull Request 7 日内首次回复；安全问题通过私有安全公告渠道报告。
持续集成	每次提交运行测试（Python 3.10、3.12）、基准、全部演示，并构建与运行容器镜像。

依赖合规	已核实直接依赖的许可证并记录来源；传递依赖尚未逐项核实。
隐私	默认离线、无遥测；报告与界面对长数字与邮箱遮罩；审计日志不记录被删内容；删除请求完成后清除工作区内的目标值副本；对 Mem0 默认遥测强制关闭。无法覆盖 shell 历史、备份、副本等工作区之外的位置。
维护者	【待补：维护者名单与联系方式；目前 GOVERNANCE 中仅列项目发起人】

11 商业模式与长期规划

11.1 开放核心模式

开源版（Apache-2.0，完整可运行）	商业版（仅为路线图设想，比赛期间未实现）
数据模型、适配器、调查器、修正器、验证器、残留探测、命令行、Web 界面、报告、示例与基准集	托管控制台、持续监控与告警、企业系统连接器、行业合规报告模板、单点登录与权限、技术支持与服务等级

目前没有付费客户，也没有收入；本节只说明开源版与未来可能的商业能力的边界，不含任何市场规模或收入预测（无可核实依据）。

11.2 路线图

版本	时间	内容
v0.1	2026-10	已完成：溯源、调查（M1–M6）、修正、验证、证据报告、命令行与界面；JSON、SQLite、Mem0 适配器；memwall 导入；仿真基准与真实堆栈验证。
v0.2	2026-12（计划）	Letta 适配器的真实模型与嵌入器验证、核心记忆块与会话消息；PyPI 发布；Windows 与 macOS 持续集成；HTML 报告；从 OpenTelemetry 追踪导入事故。
v0.3	2027 年一季度（计划）	持续监控；更多探测方法；多智能体共享记忆的作用域检查；更多嵌入模型的验证。
v1.0	2027 年（计划）	稳定接口；社区适配器；基于更真实数据的验证。

12 团队与资源

项目	内容
团队名称与成员	【待补】
分工与时间投入	【待补】
维护能力与长期投入承诺	【待补】
联系方式	【待补】

本节内容尚未提供，不作任何假设性填写。

13　风险与对策

风险	对策
与已有项目功能重叠，被认为重复造轮子	逐项核实并如实列出对方已有能力（第 7 节）；创新点表述为组合与流程；与 memwall 互补而不重做扫描。
在接入真实系统的广度上落后于 Ferryte 等	先做 Mem0 与 SQLite 的真实验证并公开限制；其余适配器按实际可验证性推进，不交付未经运行的适配器。
仿真数据结论被过度解读	文档与基准结果处处标明「仿真」「样本小」；性能数字只来自实际运行输出。
语言模型输出不稳定	默认离线、确定性；验证器对空白与排版不敏感，并以规则抽取交叉判断；记录失败批次，不只留成功批次。
上游接口变动（Mem0、Letta 等）	适配器隔离；参考实现不依赖外部框架；可选依赖固定到已测试的小版本，适配器启动时对未测试的 Mem0 版本发出警告。
名称与已有项目混淆	已改名为 Riffnel_memguard，并在 README 中说明。
个人信息与密钥泄露	默认离线无遥测；报告遮罩；审计日志不含被删内容；密钥只通过环境变量传入，不写入仓库。

附录

附录 A　命令速查

命令	作用
`riffnel-memguard demo stale | erasure`	离线演示：条款过期、删除残留
`riffnel-memguard demo sqlite-stale | sqlite-erasure`	同上，使用 SQLite 存储
`riffnel-memguard demo real-stale | real-erasure`	真实堆栈（Mem0 + Pydantic AI）；可用环境变量接真实模型与嵌入器
`riffnel-memguard demo letta-stale | letta-erasure`	真实 Letta 服务端（需先运行，见 scripts/letta_test_env.sh）
`init / ask / investigate / plan / diff / approve / apply / verify / report`	分步流程
`link / impact / scan / revalidate`	溯源链接、影响分析、记忆体检、来源更新重验
`erase、export-memwall、import-memwall`	删除请求、与 memwall 配合
`benchmark、audit verify、ui`	基准、审计链校验、Web 界面

附录 B　参考来源

- Mem0：https://github.com/mem0ai/mem0（README 与 2.2.1 版接口实测）
- Letta：https://github.com/letta-ai/letta
- memwall：https://github.com/gbayerv26/memwall（README 与 0.6.1 版输出实测）
- typed-recall：https://github.com/yash194/recall
- Aegis Memory：https://github.com/quantifylabs/aegis-memory
- memory-blackbox：https://github.com/lavkumarv/agent-forensics
- MemGuard（ac12644）：https://github.com/ac12644/MemGuard
- Ferryte：https://github.com/getferryte/ferryte
- 同名学术论文：Jia 等，MemGuard: Defending against Black-Box Membership Inference Attacks via Adversarial Examples，ACM CCS 2019，https://arxiv.org/pdf/1909.10594（与本项目无关）
- 个人信息保护法第四十七条（二手资料，未核对正式文本）：https://prighter.com/resources/laws/chinese-pipl/articles/article-47
- OWASP 智能体应用十大风险 2026 之 ASI06（二手资料，未核对 OWASP 原文）：https://docs.modulos.ai/frameworks/owasp-top-10-agentic
- Pydantic AI：https://github.com/pydantic/pydantic-ai；fastembed：https://pypi.org/project/fastembed/；BAAI/bge-small-zh-v1.5：https://huggingface.co/BAAI/bge-small-zh-v1.5

附录 C　提交前待补事项

1. 团队名称、成员、分工、联系方式（封面与第 12 节）；
2. 演示视频链接（封面）；
3. GOVERNANCE 中的维护者名单与联系方式（第 10 节）；
4. 个人信息保护法与 OWASP 条目改为核对原文后的引用（第 2.4 节）；
5. 如在比赛截止前取得更多验证（例如更多场景或模型的真实运行），更新第 9 节并保留失败记录。